DESAFIO PUC-RIO 2026 — SIMULADO 02

MODO DIAGNÓSTICO • AVALIAÇÃO DE RACIOCÍNIO E MAPEAMENTO DE LACUNAS

(OBS) OBSERVAÇÃO IMPORTANTE — MODO DIAGNÓSTICO & GRAVAÇÃO

• **Gravação Tríplice Obrigatória:** Grave a sua **tela**, seu **áudio** (pensando em voz alta e verbalizando cada dúvida ou bloqueio) e sua **câmera** no canto da tela durante toda a resolução.

• **Sem Cronômetro:** Não se preocupe com o tempo de prova! O objetivo principal é mapearmos exatamente onde ocorrem as hesitações, dúvidas de sintaxe ou bloqueios de lógica.

• **Formatação de Código:** Escreva soluções estritamente em Python 3. Use input() sem textos explicativos no prompt e exiba a saída exatamente como especificada.

• **Testes Obrigatórios:** Teste seu código com os 4 exemplos da tabela e crie **2 casos de teste próprios** (sendo um focado em casos de borda) antes de concluir a questão.

• **Envio e Diagnóstico:** Ao terminar, envie o vídeo (ex.: link do YouTube Não Listado) e os seus códigos em Python 3. As dificuldades mapeadas gerarão um **Guia Explicativo em PDF** e uma **Lista de Reforço Personalizada**.

Questão 1 — Detecção de Hotspots e Submatriz de Soma Máxima

Descrição: Uma empresa de monitoramento ambiental analisa uma matriz N x M representando níveis de radiação em uma região. Sua tarefa é identificar submatrizes quadradas de tamanho K x K cuja soma de valores ultrapasse um limiar de alerta L. Você deve contar quantas submatrizes de tamanho K x K possuem soma estritamente maior que L e informar as coordenadas (linha e coluna, base 0) do canto superior esquerdo da submatriz que possui a **maior soma absoluta**. Em caso de empate na maior soma, escolha a submatriz que aparecer primeiro na varredura (menor linha e, em seguida, menor coluna).

Entrada: A primeira linha contém três inteiros N, M e K (K <= N, K <= M). As próximas N linhas contêm M inteiros cada. A última linha contém o inteiro L.

Saída: Exiba duas linhas: a primeira com a quantidade de submatrizes K x K com soma > L (ex: 'HOTSPOTS: X'); a segunda com as coordenadas do canto superior esquerdo da submatriz de maior soma (ex: 'MAIOR: L C SOMA'). Se nenhuma ultrapassar L, informe a de maior soma mesmo assim.

Ex.	Entrada	Saída Esperada	Explicação / Foco

1	`3 3 2`
	`1 2 3`
	`4 5 6`
	`7 8 9`
	`15`
		`HOTSPOTS: 3`
		`MAIOR: 1 1 28`
			Duas submatrizes 2x2 com soma > 15. A de maior soma é no canto (1,1) com soma 28.

2	`3 4 2`
	`10 10 0 0`
	`10 10 0 0`
	`0 0 0 0`
	`30`
		`HOTSPOTS: 1`
		`MAIOR: 0 0 40`
			Apenas a submatriz 2x2 no topo esquerdo soma 40 > 30.

3	`2 2 2`
	`1 1`
	`1 1`
	`10`
		`HOTSPOTS: 0`
		`MAIOR: 0 0 4`
			Nenhuma soma ultrapassa L=10. Exibe HOTSPOTS: 0 e a única submatriz.

Ex.	Entrada	Saída Esperada	Explicação / Foco

4 (Borda)	`4 4 1`
	`5 2 8 1`
	`3 9 4 6`
	`7 1 2 5`
	`0 0 0 0`
	`4`
		`HOTSPOTS: 6`
		`MAIOR: 1 1 9`
			K=1 analisa elementos individuais. Maior elemento é 9 no índice (1,1).

Antes de programar, pense:

1. Como calcular a soma de uma submatriz K x K a partir da posição (r, c) usando dois loops aninhados?
2. Qual a condição para garantir que a janela K x K não saia dos limites da matriz N x M?
3. Como manter o rastreamento da maior soma encontrada e aplicar a regra de desempate corretamente?

Questão 2 — Simulação de Histórico de Ações e Mecanismo Undo/Redo

Descrição: Em um editor de texto simplificado, as ações do usuário são registradas em uma lista. As ações possíveis são: DIGITAR [palavra] (adiciona uma palavra ao documento), UNDO (desfaz a última palavra digitada e a move para a pilha de refazer) e REDO (restaura a última palavra desfeita). Se o usuário fizer uma nova ação DIGITAR, o histórico de REDO deve ser totalmente limpo. Você deve processar N comandos e, ao final, exibir o estado final do texto (palavras separadas por espaço). Se o texto final estiver vazio, exiba 'TEXTO VAZIO'.

Entrada: A primeira linha contém um inteiro N (quantidade de comandos). As próximas N linhas contêm um comando cada ('DIGITAR x', 'UNDO' ou 'REDO').

Saída: Uma única linha com o texto final composto pelas palavras separadas por espaço, ou 'TEXTO VAZIO'.

Ex.	Entrada	Saída Esperada	Explicação / Foco

1	`5`
	`DIGITAR Ola`
	`DIGITAR Mundo`
	`UNDO`
	`DIGITAR Python`
	`REDO`
		`Ola Python`	O UNDO remove 'Mundo'. DIGITAR 'Python' limpa a pilha de REDO, tornando o REDO final inválido.

2	`4`
	`DIGITAR PUC`
	`UNDO`
	`REDO`
	`REDO`
		`PUC`	O segundo REDO é ignorado pois não há nada para refazer.

3	`3`
	`DIGITAR Teste`
	`UNDO`
	`UNDO`
		`TEXTO VAZIO`	O segundo UNDO é ignorado pois o texto já estava vazio.

4 (Borda)	`6`
	`DIGITAR A`
	`DIGITAR B`
	`DIGITAR C`
	`UNDO`
	`UNDO`
	`REDO`
		`A B`	Desfaz C e B. Refaz B. Texto final fica 'A B'.

Antes de programar, pense:
1. Quais estruturas de dados (listas em Python) você usará para guardar o texto atual e a pilha de refazer (redo)?
2. O que acontece com a lista de redo quando um novo comando 'DIGITAR' é executado?
3. Como tratar o comando 'UNDO' quando o documento já está totalmente vazio?

Questão 3 — Agrupamento e Filtragem de Anagramas

Descrição: Duas palavras são anagramas se contêm exatamente as mesmas letras com as mesmas frequências, ignorando maiúsculas e minúsculas. Dada uma lista de N palavras, você deve identificar todas as palavras que possuem **pelo menos um anagrama** presente na lista. Em seguida, agrupe essas palavras em ordem alfabética (convertidas para minúsculas e sem duplicatas) e exiba a lista resultante. Se nenhuma palavra possuir anagramas na lista, exiba 'NENHUM ANAGRAMA'.

Entrada: A primeira linha contém um inteiro N. As próximas N linhas contêm uma palavra simples (sem espaços) cada.

Saída: As palavras que possuem anagramas na lista, ordenadas alfabeticamente, separadas por espaço (em minúsculas). Se não houver, exiba 'NENHUM ANAGRAMA'.

Ex.	Entrada	Saída Esperada	Explicação / Foco

1	`4`
	`Amor`
	`Roma`
	`Casa`
	`Ramo`
		`amor ramo roma`	'Amor', 'Roma' e 'Ramo' são anagramas entre si. 'Casa' não tem anagramas na lista.

2	`3`
	`Python`
	`Java`
	`C`
		`NENHUM ANAGRAMA`	Nenhuma palavra possui anagrama entre as fornecidas.

3	`4`
	`Pedra`
	`PERDA`
	`pedra`
	`Gato`
		`pedra perda`	Comparações são case-insensitive. Duplicatas idênticas devem ser tratadas sem repetir.

4 (Borda)	`2`
	`abc`
	`cba`
		`abc cba`	Duas palavras simples que são anagramas perfeitos.

Antes de programar, pense:
1. Como testar se duas palavras são anagramas usando a ordenação de seus caracteres (`sorted(p.lower())`)?
2. Como evitar adicionar palavras duplicadas no resultado final?
3. Como garantir que palavras sem nenhum parceiro anagrama sejam descartadas da saída?

Questão 4 — Validador de Sequência e Dependência de Tarefas

Descrição: Um sistema de build precisa executar N tarefas numeradas de 0 a N-1. Existem M regras de dependência no formato 'A B', significando que a tarefa A **precisa ser concluída antes** da tarefa B. O usuário fornece uma ordem proposta de execução contendo as N tarefas. Sua missão é verificar se a ordem proposta respeita todas as M regras de

dependência. Se a ordem for totalmente válida, exiba 'ORDEM VALIDA'. Caso contrário, exiba 'ORDEM INVALIDA'.

Entrada: A primeira linha contém N e M. As próximas M linhas contêm dois inteiros A e B. A última linha contém N inteiros representando a ordem proposta.

Saída: 'ORDEM VALIDA' ou 'ORDEM INVALIDA'.

Ex.	Entrada	Saída Esperada	Explicação / Foco

1	`3 2`
	`0 1`
	`1 2`
	`0 1 2`
		`ORDEM VALIDA`	0 vem antes de 1, e 1 vem antes de 2. A ordem proposta 0 1 2 respeita tudo.

2	`3 2`
	`0 1`
	`1 2`
	`2 1 0`
		`ORDEM INVALIDA`	A regra exige 0 antes de 1 e 1 antes de 2, mas a proposta colocou 2 antes.

3	`4 1`
	`2 0`
	`1 3 2 0`
		`ORDEM VALIDA`	A única regra é 2 antes de 0. Na sequência '1 3 2 0', o 2 aparece no índice 2 e o 0 no índice 3.

4 (Borda)	`2 0` `1 0`	`ORDEM VALIDA`	Sem regras de dependência (M=0), qualquer ordem contendo todas as tarefas é válida.

Antes de programar, pense:
1. Como descobrir a posição (índice) de cada tarefa na lista de ordem proposta?
2. Como verificar se `posicao[A] < posicao[B]` para todas as M regras?
3. Qual estrutura de controle permite interromper a verificação assim que a primeira violação for encontrada?

Questão 5 — Compactador de Matrizes Esparsas e Cálculo de Eficiência

Descrição: Uma matriz esparsa é uma matriz na qual a maioria dos elementos é igual a zero. Para economizar memória, podemos armazenar apenas os elementos **não-nulos** no formato triplet: (linha, coluna, valor). Dada uma matriz N x M, você deve gerar a representação compactada apenas com os elementos diferentes de zero (em ordem de varredura por linhas e colunas) e calcular o percentual de economia de elementos armazenados. A economia é calculada como: Economia = ((Total_Elementos_Original - Elementos_Nao_Nulos) / Total_Elementos_Original) * 100. Formatado com 1 casa decimal.

Entrada: A primeira linha contém N e M. As próximas N linhas contêm M inteiros cada.

Saída: Para cada elemento não-nulo, imprima 'L C V'. Na última linha, imprima 'ECONOMIA: X.X%'.

Ex.	Entrada	Saída Esperada	Explicação / Foco

1	`3 3`
	`0 0 5`
	`0 2 0`
	`7 0 0`
		`0 2 5`
		`1 1 2`
		`2 0 7`
		`ECONOMIA: 66.7%`
			Original tinha 9 elementos. Não-nulos são 3. Economia = (6/9)*100 = 66.7%.

Ex.	Entrada	Saída Esperada	Explicação / Foco

2	`2 2`
	`1 2`
	`3 4`
		`0 0 1`
		`0 1 2`
		`1 0 3`
		`1 1 4`
		`ECONOMIA: 0.0%`
			Nenhum elemento nulo. Economia de memória é 0.0%.

3	`2 3`
	`0 0 0`
	`0 0 0`
		`ECONOMIA: 100.0%`	Todos elementos nulos. Nenhuma linha 'L C V' é impressa antes do percentual.

4 (Borda)	`1 4` `0 9 0 0`	`0 1 9` `ECONOMIA: 75.0%`	Apenas 1 elemento não nulo em 4. Economia = (3/4)*100 = 75.0%.

Antes de programar, pense:

1. Como percorrer a matriz N x M salvando apenas os tripletos `(i, j, matriz[i][j])` quando o valor for diferente de 0?
2. Como calcular o total de elementos da matriz original (`N * M`) e a porcentagem de zeros com precisão de float?
3. Como formatar a saída com uma casa decimal usando f-string (`f'{economia:.1f}%'`)?